1. Implement and demonstrate the FIND-S algorithm for finding the most specific hypothesis based on a given set of training data samples.

```
import matplotlib.pyplot as plt

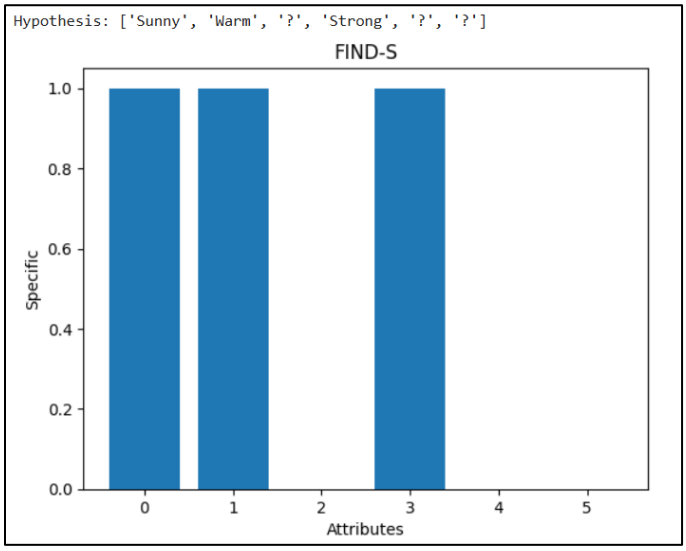
data = [
    ['Sunny', 'Warm', 'Normal', 'Strong', 'Warm', 'Same', 'Yes'],
    ['Sunny', 'Warm', 'High', 'Strong', 'Warm', 'Same', 'Yes'],
    ['Rainy', 'Cold', 'High', 'Strong', 'Warm', 'Change', 'No'],
    ['Sunny', 'Warm', 'High', 'Strong', 'Cool', 'Change', 'Yes']
]

h = ['0', '0', '0', '0', '0', '0']

for row in data:
    if row[6] == 'Yes':
        for i in range(6):
            if h[i] == '0':
                h[i] = row[i]
            elif h[i] != row[i]:
                h[i] = '?'

print("Hypothesis:", h)

plt.bar(range(6), [1 if x != '?' else 0 for x in h])
plt.xlabel("Attributes")
plt.ylabel("Specific")
plt.title("FIND-S")
plt.show()
```



2. For a given set of training data examples stored in a .CSV file, implement and demonstrate the Candidate-Elimination algorithm in python to output a description of the set of all hypotheses consistent with the training examples

```
import matplotlib.pyplot as plt

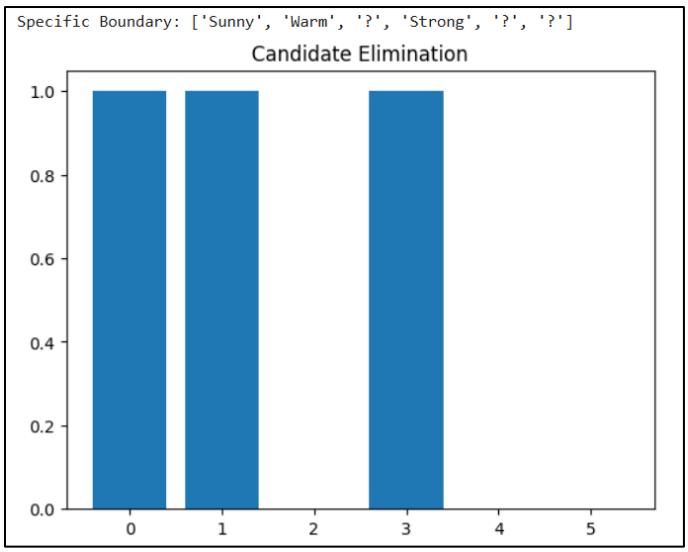
data = [
    ['Sunny', 'Warm', 'Normal', 'Strong', 'Warm', 'Same', 'Yes'],
    ['Sunny', 'Warm', 'High', 'Strong', 'Warm', 'Same', 'Yes'],
    ['Rainy', 'Cold', 'High', 'Strong', 'Warm', 'Change', 'No'],
    ['Sunny', 'Warm', 'High', 'Strong', 'Cool', 'Change', 'Yes']
]

S = ['0', '0', '0', '0', '0', '0']

for row in data:
    if row[6] == 'Yes':
        for i in range(6):
            if S[i] == '0':
                S[i] = row[i]
            elif S[i] != row[i]:
                S[i] = '?'

print("Specific Boundary:", S)

plt.bar(range(6), [1 if x != '?' else 0 for x in S])
plt.title("Candidate Elimination")
plt.show()
```



3. Demonstrate the working of the decision tree based ID3 algorithm. Use an appropriate data set for building the decision tree and apply this knowledge to classify a new sample.

```
from sklearn.datasets import load iris
from sklearn.tree import DecisionTreeClassifier, plot_tree
import matplotlib.pyplot as plt

iris = load iris()

X = iris.data
y = iris.target

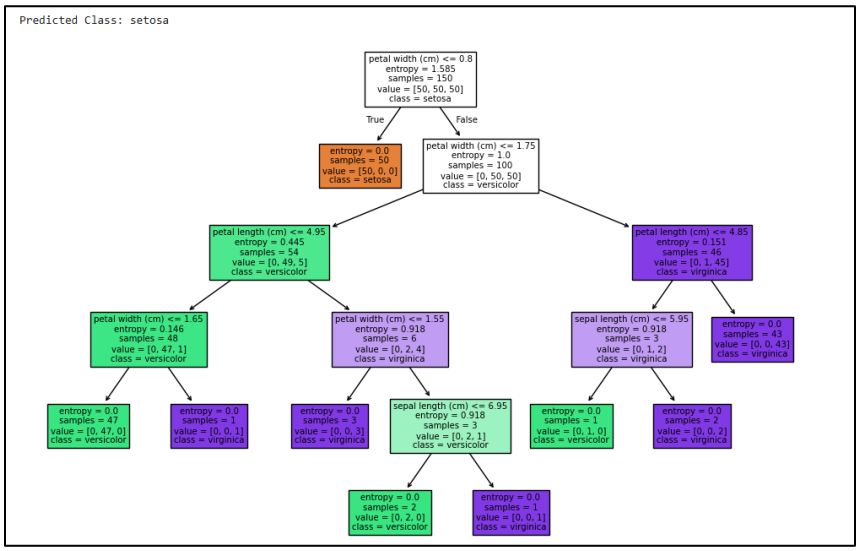
model = DecisionTreeClassifier(criterion='entropy', random state=1)

model.fit(X, y)

prediction = model.predict([[5.1, 3.5, 1.4, 0.2]])

print("Predicted Class:", iris.target names[prediction][0])

plt.figure(figsize=(12, 8))
plot tree(model, feature names=iris.feature names,
          class names=iris.target names, filled=True)
plt.show()
```



4. Build an Artificial Neural Network by implementing the Backpropagation algorithm and test the same using appropriate data sets.

```
from sklearn.neural network import MLPClassifier
import matplotlib.pyplot as plt

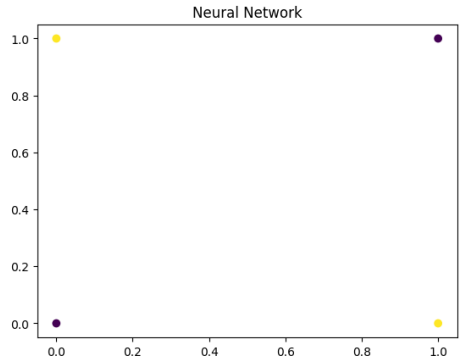
X = [[0,0],[0,1],[1,0],[1,1]]
y = [0,1,1,0]

model = MLPClassifier(hidden layer sizes=(2,), max iter=1000)
model.fit(X,y)

print("Prediction:", model.predict([[1,1]]))

plt.scatter([0,0,1,1],[0,1,0,1],c=y)
plt.title("Neural Network")
plt.show()
```

```
/usr/local/lib/python3.13/dist-packages/sklearn/neural_network/_multilayer_perceptron.py:691: ConvergenceWarning: Stochastic Optimizer: Maximum iterations (1000) reached and the optimization hasn't converged yet.
  warnings.warn(
Prediction: [0]
```



5. Write a program for Implementation of K-Nearest Neighbours (K-NN) in Python

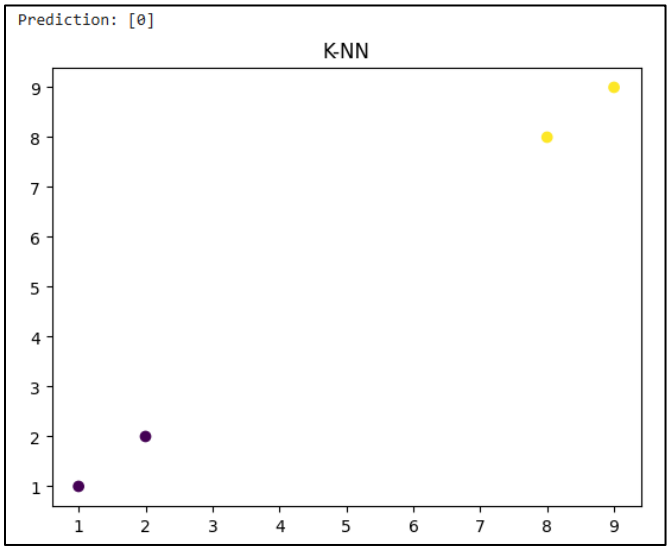
```
from sklearn.neighbors import KNeighborsClassifier
import matplotlib.pyplot as plt

X = [[1,1],[2,2],[8,8],[9,9]]
y = [0,0,1,1]

model = KNeighborsClassifier(3)
model.fit(X,y)

print("Prediction:", model.predict([[3,3]]))

plt.scatter([1,2,8,9],[1,2,8,9],c=y)
plt.title("K-NN")
plt.show()
```



6. Write a program to implement Naïve Bayes algorithm in python and to display the results using confusion matrix and accuracy.

```
from sklearn.naive bayes import GaussianNB
import matplotlib.pyplot as plt

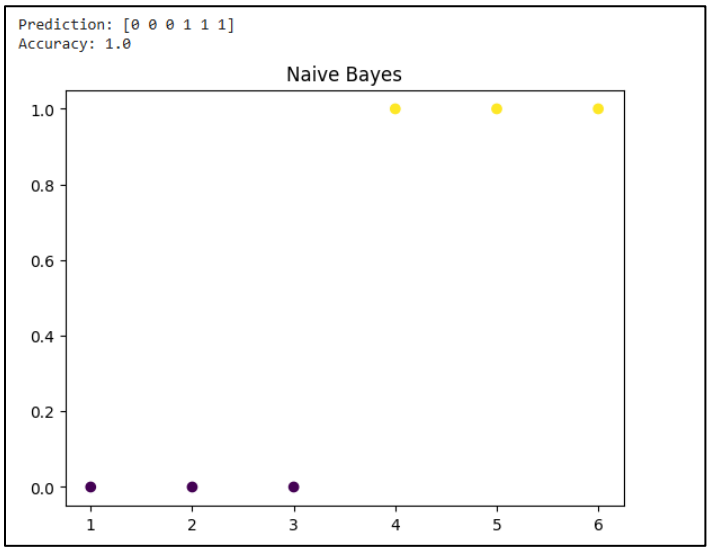
X = [[1],[2],[3],[4],[5],[6]]
y = [0,0,0,1,1,1]

model = GaussianNB()
model.fit(X,y)

p = model.predict(X)

print("Prediction:", p)
print("Accuracy:", model.score(X,y))

plt.scatter(X,y,c=p)
plt.title("Naive Bayes")
plt.show()
```



7. Write a program to implement Logistic Regression (LR) algorithm in python

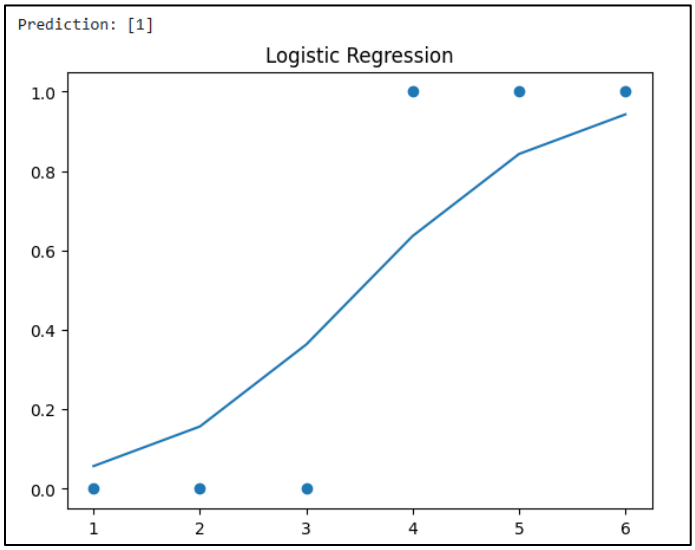
```
from sklearn.linear model import LogisticRegression
import matplotlib.pyplot as plt

X = [[1],[2],[3],[4],[5],[6]]
y = [0,0,0,1,1,1]

model = LogisticRegression()
model.fit(X,y)

print("Prediction:", model.predict([[5]]))

plt.scatter(X,y)
plt.plot(X,model.predict proba(X)[:,1])
plt.title("Logistic Regression")
plt.show()
```



8. Write a program to implement Linear Regression (LR) algorithm in python

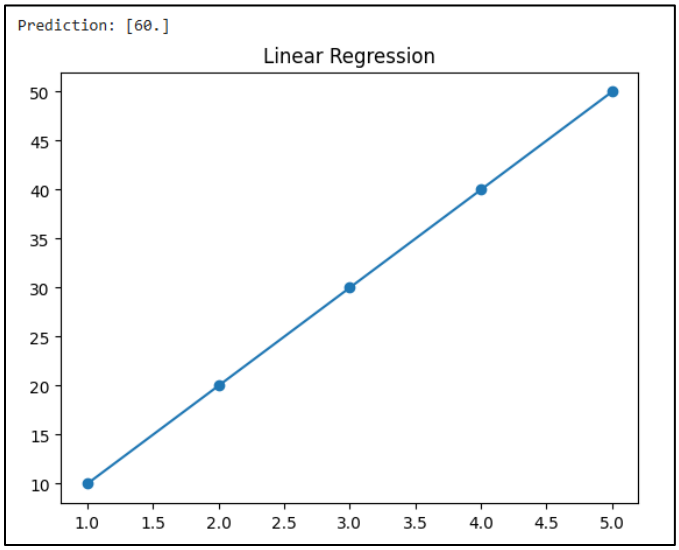
```
from sklearn.linear model import LinearRegression
import matplotlib.pyplot as plt

X = [[1],[2],[3],[4],[5]]
y = [10,20,30,40,50]

model = LinearRegression()
model.fit(X,y)

print("Prediction:", model.predict([[6]]))

plt.scatter(X,y)
plt.plot(X,model.predict(X))
plt.title("Linear Regression")
plt.show()
```



9. Compare Linear and Polynomial Regression using Python

```
from sklearn.linear model import LinearRegression
from sklearn.preprocessing import PolynomialFeatures
import matplotlib.pyplot as plt

X = [[1],[2],[3],[4],[5]]
y = [1,4,9,16,25]

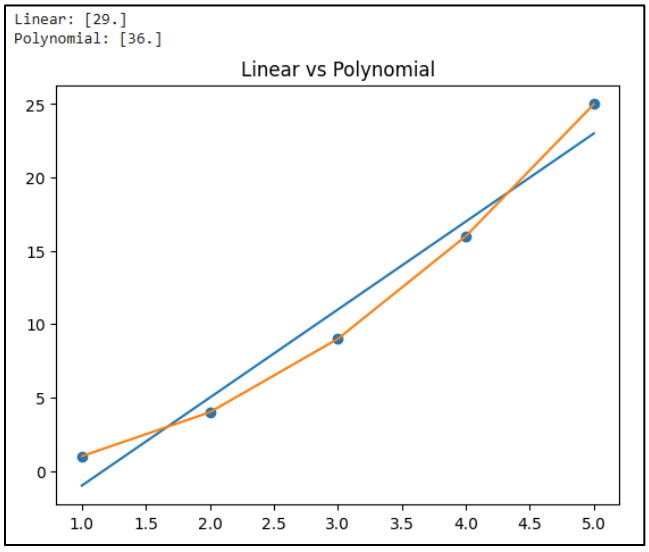
L = LinearRegression()
L.fit(X,y)

P = PolynomialFeatures(2)
XP = P.fit_transform(X)

PL = LinearRegression()
PL.fit(XP,y)

print("Linear:", L.predict([[6]]))
print("Polynomial:", PL.predict(P.transform([[6]])))

plt.scatter(X,y)
plt.plot(X,L.predict(X))
plt.plot(X,PL.predict(XP))
plt.title("Linear vs Polynomial")
plt.show()
```



10. Write a Python Program to Implement Expectation & Maximization Algorithm

```
from sklearn.mixture import GaussianMixture
import matplotlib.pyplot as plt

X = [[1],[2],[3],[10],[11],[12]]

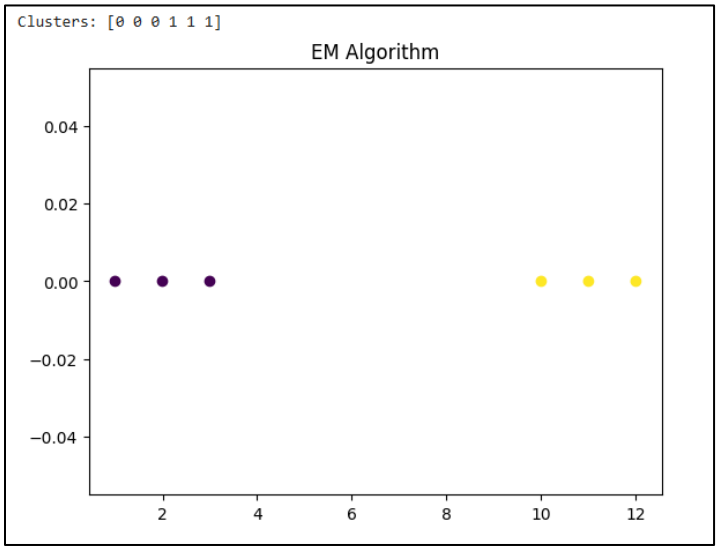
model = GaussianMixture(2)
model.fit(X)

p = model.predict(X)

print("Clusters:", p)

plt.scatter([1,2,3,10,11,12],
            [0,0,0,0,0,0], c=p)

plt.title("EM Algorithm")
plt.show()
```



11. Write a program for the task of Credit Score Classification

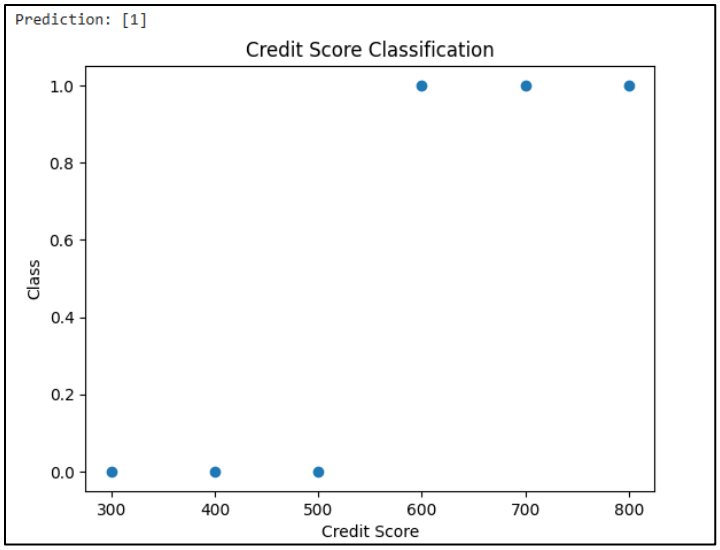
```
from sklearn.linear_model import LogisticRegression
import matplotlib.pyplot as plt

X = [[300],[400],[500],[600],[700],[800]]
y = [0,0,0,1,1,1]

model = LogisticRegression()
model.fit(X, y)

print("Prediction:", model.predict([[650]]))

plt.scatter(X, y)
plt.title("Credit Score Classification")
plt.xlabel("Credit Score")
plt.ylabel("Class")
plt.show()
```



12. Implement Iris Flower Classification using KNN

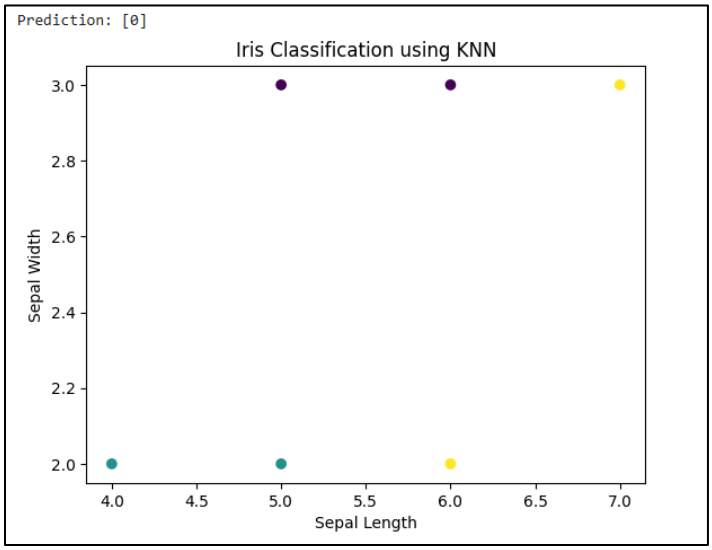
```
from sklearn.neighbors import KNeighborsClassifier
import matplotlib.pyplot as plt

X = [[5,3],[6,3],[4,2],[7,3],[5,2],[6,2]]
y = [0,0,1,2,1,2]

model = KNeighborsClassifier(3)
model.fit(X, y)

print("Prediction:", model.predict([[5,3]]))

plt.scatter([5,6,4,7,5,6], [3,3,2,3,2,2], c=y)
plt.title("Iris Classification using KNN")
plt.xlabel("Sepal Length")
plt.ylabel("Sepal Width")
plt.show()
```



13. Implement the Car Price Prediction Model using Python

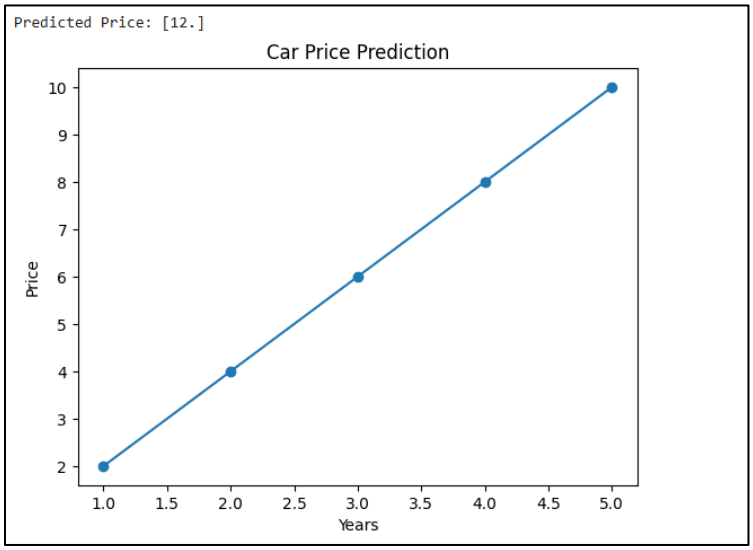
```
from sklearn.linear model import LinearRegression
import matplotlib.pyplot as plt

X = [[1],[2],[3],[4],[5]]
y = [2,4,6,8,10]

model = LinearRegression()
model.fit(X, y)

print("Predicted Price:", model.predict([[6]]))

plt.scatter(X, y)
plt.plot(X, model.predict(X))
plt.title("Car Price Prediction")
plt.xlabel("Years")
plt.ylabel("Price")
plt.show()
```



14. Implement House price Prediction using appropriate machine learning algorithm

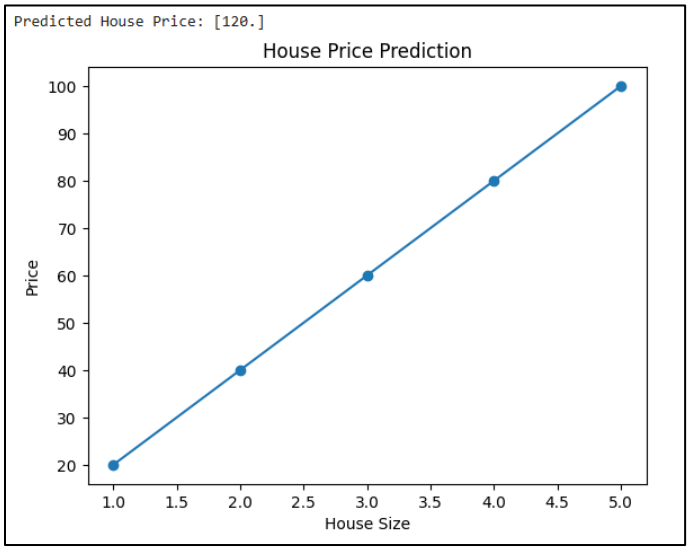
```
from sklearn.linear model import LinearRegression
import matplotlib.pyplot as plt

X = [[1],[2],[3],[4],[5]]
y = [20,40,60,80,100]

model = LinearRegression()
model.fit(X, y)

print("Predicted House Price:", model.predict([[6]]))

plt.scatter(X, y)
plt.plot(X, model.predict(X))
plt.title("House Price Prediction")
plt.xlabel("House Size")
plt.ylabel("Price")
plt.show()
```



15. Implement Iris Flower Classification using Naive Bayes classifier

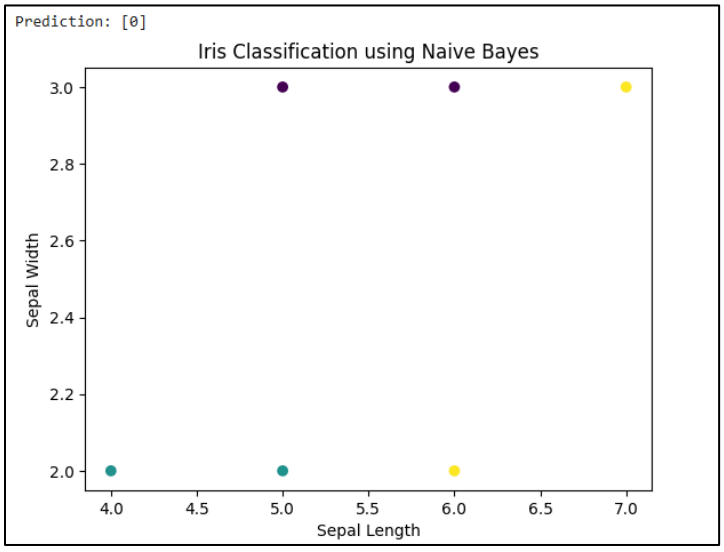
```
from sklearn.naive bayes import GaussianNB
import matplotlib.pyplot as plt

X = [[5,3],[6,3],[4,2],[7,3],[5,2],[6,2]]
y = [0,0,1,2,1,2]

model = GaussianNB()
model.fit(X, y)

print("Prediction:", model.predict([[5,3]]))

plt.scatter([5,6,4,7,5,6], [3,3,2,3,2,2], c=y)
plt.title("Iris Classification using Naive Bayes")
plt.xlabel("Sepal Length")
plt.ylabel("Sepal Width")
plt.show()
```



16. Compare different types Classification Algorithms and evaluate their performance.

```
from sklearn.linear model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier
from sklearn.naive bayes import GaussianNB
import matplotlib.pyplot as plt

X = [[1],[2],[3],[4],[5],[6]]
y = [0,0,0,1,1,1]

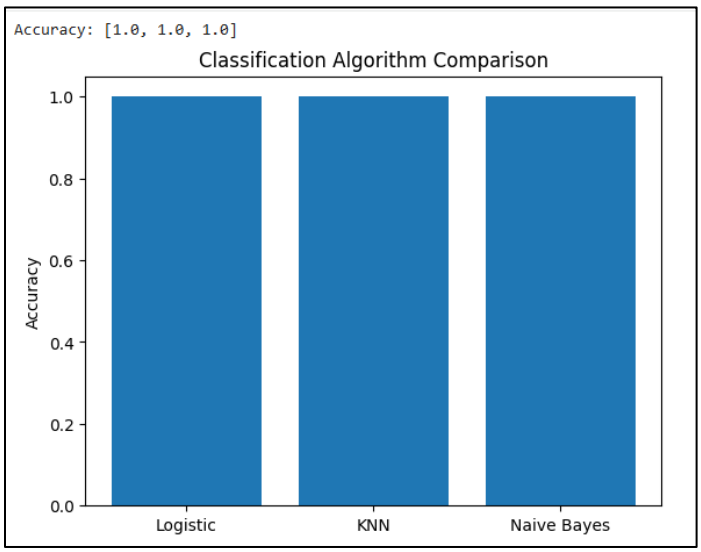
models = [
    LogisticRegression(),
    KNeighborsClassifier(3),
    GaussianNB()
]

names = ["Logistic", "KNN", "Naive Bayes"]
accuracy = []

for model in models:
    model.fit(X, y)
    accuracy.append(model.score(X, y))

print("Accuracy:", accuracy)

plt.bar(names, accuracy)
plt.title("Classification Algorithm Comparison")
plt.ylabel("Accuracy")
plt.show()
```



17. Implement Mobile Price Prediction using appropriate machine learning algorithm

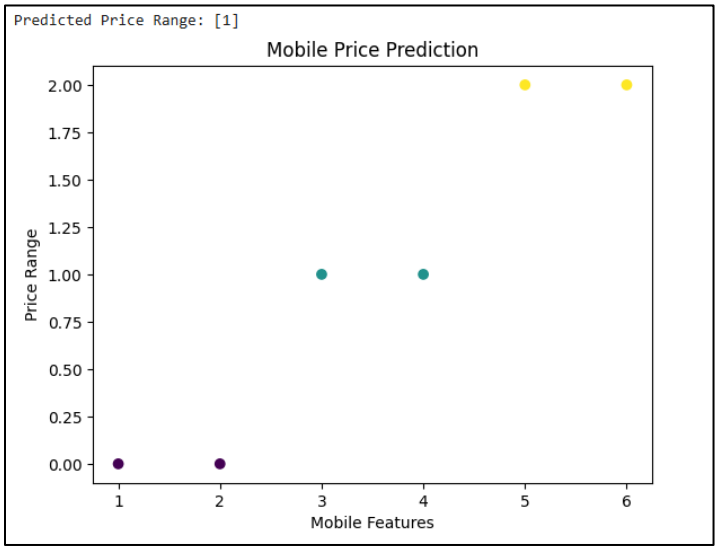
```
from sklearn.ensemble import RandomForestClassifier
import matplotlib.pyplot as plt

X = [[1],[2],[3],[4],[5],[6]]
y = [0,0,1,1,2,2]

model = RandomForestClassifier(n estimators=10, random state=1)
model.fit(X, y)

print("Predicted Price Range:", model.predict([[4]]))

plt.scatter([1,2,3,4,5,6], y, c=y)
plt.title("Mobile Price Prediction")
plt.xlabel("Mobile Features")
plt.ylabel("Price Range")
plt.show()
```



18. Implement Perceptron based IRIS classification

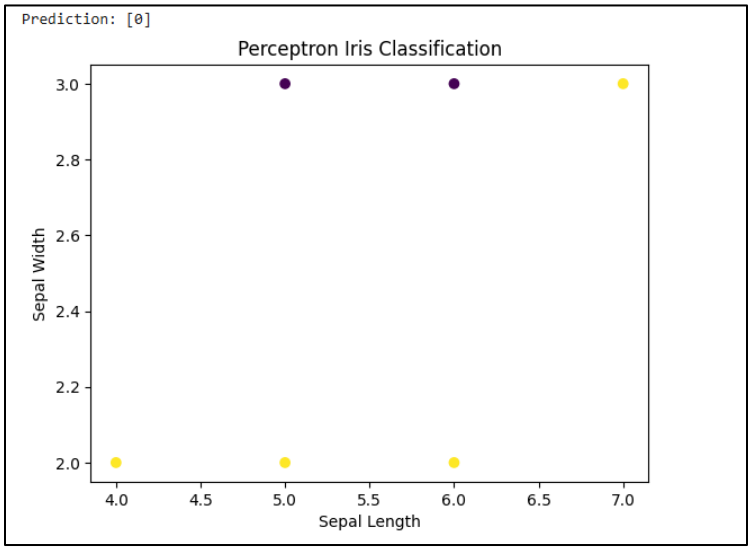
```
from sklearn.linear model import Perceptron
import matplotlib.pyplot as plt

X = [[5,3],[6,3],[4,2],[7,3],[5,2],[6,2]]
y = [0,0,1,1,1,1]

model = Perceptron()
model.fit(X, y)

print("Prediction:", model.predict([[5,3]]))

plt.scatter([5,6,4,7,5,6], [3,3,2,3,2,2], c=y)
plt.title("Perceptron Iris Classification")
plt.xlabel("Sepal Length")
plt.ylabel("Sepal Width")
plt.show()
```



19. Implementation of Naive Bayes classification for Bank Loan prediction

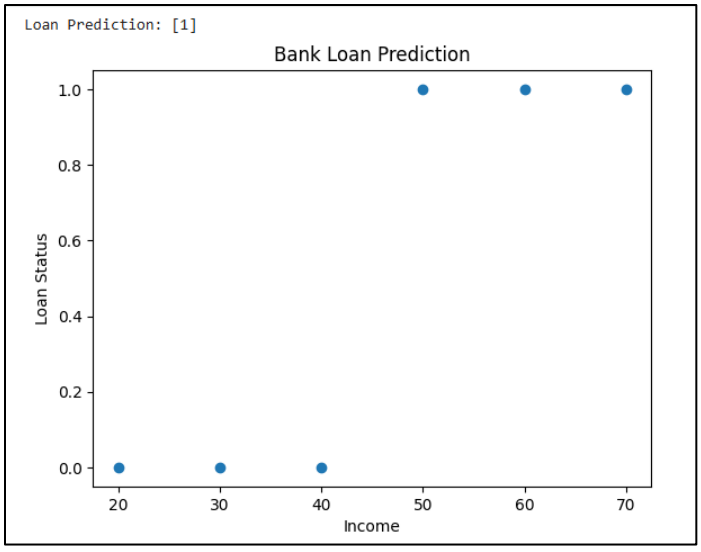
```
from sklearn.naive_bayes import GaussianNB
import matplotlib.pyplot as plt

X = [[20],[30],[40],[50],[60],[70]]
y = [0,0,0,1,1,1]

model = GaussianNB()
model.fit(X, y)

print("Loan Prediction:", model.predict([[55]]))

plt.scatter(X, y)
plt.title("Bank Loan Prediction")
plt.xlabel("Income")
plt.ylabel("Loan Status")
plt.show()
```



20. Implement Future Sales Prediction using a suitable machine learning algorithm

```
from sklearn.linear model import LinearRegression
import matplotlib.pyplot as plt

X = [[1],[2],[3],[4],[5]]
y = [100,200,300,400,500]

model = LinearRegression()
model.fit(X, y)

print("Future Sales:", model.predict([[6]]))

plt.scatter(X, y)
plt.plot(X, model.predict(X))
plt.title("Future Sales Prediction")
plt.xlabel("Month")
plt.ylabel("Sales")
plt.show()
```

